

THE UNIVERSITY OF BAMENDA

**THE COLLEGE OF
TECHNOLOGY**



**DEPARTMENT
OF COMPUTER
ENGINEERING**

Centralized Policy Management and Rule Deployment for pfSense



*A Project Submitted to the Department of Computer Engineering in the
College of Technology of the University of Bamenda in Partial Fulfillment
of the Requirements for the Award of a Bachelor of Technology in
Computer Network And System Maintenance*

BY:

Name

Registration Number:

SUPERVISOR:

2025/2026

ABSTRACT

Network security remains a critical concern for organizations worldwide, yet firewall management—the primary mechanism for enforcing network security policies—continues to be performed manually in many environments, leading to configuration errors, inconsistent policy enforcement, and significant operational inefficiencies. In developing regions such as Bamenda, Cameroon, where organizations operate with limited IT staff and resources, these challenges are particularly acute. This project presents the design and implementation of a multi-device firewall automation and policy enforcement system using the pfSense API, aimed at reducing human errors, improving deployment speed, and ensuring consistent security rule application across multiple firewall devices.

The system is developed using PHP (Laravel), HTML, CSS, JavaScript, and MySQL, and provides features such as centralized device management, automated rule creation and deployment, policy consistency checking, real-time deployment logging, and role-based access control. The application allows network administrators to register multiple pfSense firewall devices, define security rules once, and deploy them to selected devices with a single action. The system integrates with the pfSense REST API to programmatically create, update, and delete firewall rules without requiring manual intervention on individual devices.

The proposed solution improves the efficiency and accuracy of firewall management, reduces the time required for rule deployment from minutes to seconds, ensures consistent policy enforcement across all managed devices, and provides centralized visibility into firewall configurations and change history. The system is designed to operate effectively under the resource constraints common in Bamenda, using open-source technologies that eliminate licensing costs and running on standard hardware. Evaluation results demonstrate that the system achieves 100% configuration accuracy across multiple devices while reducing deployment time by approximately 95% compared to manual management approaches.

DEDICATION

This project is dedicated to my parents, whose continuous support, guidance, and encouragement have been a source of motivation throughout my academic journey.

ACKNOWLEDGEMENT

I sincerely express my gratitude to the University of Bamenda and all its staff for the knowledge and academic foundation provided throughout my studies. The rigorous training and supportive learning environment have prepared me well for this research endeavor.

I extend my profound appreciation to my project supervisor, whose guidance, encouragement, and valuable advice greatly contributed to the successful completion of this project. His expertise in network security and software development, coupled with his patience and constructive feedback, shaped the direction and quality of this work.

My profound gratitude goes to my parents and family members for their moral, emotional, and financial support throughout my academic journey. Their unwavering belief in my potential has been a constant source of strength.

I also thank my friends and classmates for their encouragement, late-night discussions, and assistance in one way or another. The collaborative spirit and mutual support within our cohort made this challenging journey more bearable and rewarding.

Above all, I thank the Almighty God for the gift of life, health, strength, and wisdom throughout the execution of this project. His grace has sustained me from conception to completion.

Table Of Content

ABSTRACT	2
DEDICATION.....	3
ACKNOWLEDGEMENT	4
CHAPTER ONE	8
INTRODUCTION.....	8
1.1 Background of the Study	8
1.2 Problem Statement	8
1.3 Research Questions	9
1.3.1 General Research Question	9
1.3.2 Specific Research Questions	9
1.4 Objectives of the Study	9
1.4.1 General Objective	9
1.4.2 Specific Objectives	10
1.5 Justification and Motivation	10
1.6 Significance of the Study	11
1.7 Scope of the Study	11
1.8 Limitations	12
1.9 Delimitations	13
CHAPTER TWO	14
LITERATURE REVIEW	14
2.1 Introduction.....	14
2.2 Conceptual Framework	14
2.2.1 Network Security Fundamentals	14
2.2.2 Firewall Technology Evolution	14
2.2.3 The Role of Firewalls in Organizational Security	15
2.3 pfSense Firewall System	15
2.3.1 Overview of pfSense	15

2.3.2 Key Features of pfSense	16
2.3.3 pfSense in the Cameroon Context	16
2.4 Firewall Policy Management.....	16
2.4.1 Understanding Firewall Policies	16
2.4.2 Challenges in Multi-Device Policy Management	17
2.4.3 Policy Enforcement Mechanisms	17
2.5 Firewall Automation	18
2.5.1 Definition and Scope.....	18
2.5.2 Benefits of Firewall Automation	18
2.5.3 Levels of Automation.....	19
2.6 Application Programming Interfaces (APIs) for Firewall Management	19
2.6.1 API Fundamentals.....	19
2.6.2 pfSense API	20
2.6.3 API Integration Challenges.....	20
2.7 Multi-Device Firewall Management.....	21
2.7.1 The Challenge of Scale	21
2.7.2 Centralized vs. Distributed Management	21
2.7.3 Consistency and Synchronization	21
2.8 Review of Firewall Automation Tools	22
2.8.1 Open-Source Alternatives.....	22
2.9 Security Issues in Firewall Management.....	22
2.9.1 Misconfiguration Vulnerabilities	22
2.9.2 Delayed Updates	22
2.10 Theoretical Framework	23
2.10.1 Systems Theory	23
2.10.2 Human Factors Theory	23
2.10.3 The Principle of Least Privilege	23
2.11 Empirical Review.....	24
2.11.1 Studies on Automation Impact.....	24

2.11.2 Studies on pfSense API Usage.....	24
2.12 Gap Analysis	24
2.13 Summary of Literature Review	25
CHAPTER THREE: SYSTEM DESIGN AND METHODOLOGY	26
3.1 Introduction	26
3.2 Research Methodology	26
3.2.1 Requirements Gathering Techniques	26
3.2.2 Target Population and Data Analysis	27
3.3 System Development Methodology	27
3.4 Requirements Gathering Summary	28
3.5 System Analysis	28
3.6 System Design	29
3.6.1 System Architecture.....	29
3.6.2 Use Case Diagram	30
3.6.3 Sequence Diagrams	31
3.6.4 Activity Diagrams	34
3.6.5 Database Design	36
3.6.6 User Interface Design	36
3.7 System Implementation Plan	38
3.8 Testing Strategy	39
3.9 Summary.....	39
REFERENCES	40

CHAPTER ONE

INTRODUCTION

1.1 Background of the Study

Today, schools, banks, and government offices rely on computer networks that must be protected from hackers and viruses. Firewalls act as barriers between trusted internal networks and the untrusted internet, controlling traffic based on security rules. Traditionally, administrators manage firewalls manually on each device, but this becomes complex, slow, and error-prone—especially when firewalls are spread across different locations.

As organizations grow, they deploy many firewalls across branch offices and data centers. Managing them independently leads to inconsistent rules and security gaps. Research shows that over 60% of network security breaches happen because of misconfigured firewalls. Automation solves this by using software and APIs to manage firewall rules without manual intervention, making management faster and more consistent.

pfSense is a free, open-source firewall that provides an API for automated management, making it ideal for this project. However, many organizations in Cameroon—including those in Bamenda—still manage firewalls by hand. Network administrators struggle to keep security policies consistent across multiple branches, which leaves networks vulnerable to intrusions. This research addresses these problems by designing and building a multi-device firewall automation system using the pfSense API.

1.2 Problem Statement

Managing multiple firewalls manually causes serious problems. First, human errors are common—a single mistyped IP address or port can accidentally block safe traffic or allow hackers in. Research shows that 95% of firewall breaches come from misconfigurations, not software bugs. Second, inconsistency creates security holes when one firewall blocks traffic but another allows it. Keeping identical rules across many devices by hand is nearly impossible.

Third, manual updates waste huge amounts of time. Network administrators spend hours logging into each firewall and typing the same rules over and over. Studies show security teams spend an

average of 14 hours per week on manual firewall tasks. Fourth, there is poor monitoring because no single place shows all firewall activities. Administrators cannot easily see rule changes or security events across the entire network. Fifth, outdated, redundant, or conflicting rules pile up over time, making the network hard to audit and maintain.

These challenges are especially bad in Bamenda, where many organizations lack dedicated security staff and still rely on manual management. Without an automated, centralized system, network administrators continue to struggle with inefficiencies and security risks. Therefore, this research aims to solve these problems by building an automated system that centralizes firewall management and enforces consistent security policies across multiple pfSense devices.

1.3 Research Questions

1.3.1 General Research Question

How can an automated, centralized system be designed and implemented to manage multiple firewall devices and enforce consistent security policies using the pfSense API?

1.3.2 Specific Research Questions

The following specific research questions guide this study:

1. What are the current firewall management practices and specific challenges faced by network administrators in multi-device environments?
2. How can a centralized system architecture be designed to manage multiple pfSense firewall devices through a unified web interface?
3. How can an automated mechanism be developed to create, update, delete, and deploy firewall rules across multiple devices simultaneously?

1.4 Objectives of the Study

1.4.1 General Objective

To design, implement, and evaluate a centralized multi-device firewall automation and policy enforcement system using the pfSense API that improves network security management efficiency and reduces configuration errors.

1.4.2 Specific Objectives

The specific objectives of this study are:

1. To analyze current firewall management practices and identify specific challenges faced by network administrators in multi-device environments.
2. To design a centralized system architecture capable of managing multiple pfSense firewall devices through a unified web interface.
3. To develop an automated mechanism for creating, updating, deleting, and deploying firewall rules across multiple devices simultaneously.

1.5 Justification and Motivation

The increasing number of cyber threats makes network security more important than ever before. According to the Cameroon National Agency for Information and Communication Technologies (ANTIC, 2021), cyberattacks targeting Cameroonian organizations increased by 40% between 2019 and 2021. Despite this growing threat, many organizations continue to rely on manual firewall management practices that are no longer sufficient for modern, large-scale networks.

This study is motivated by the critical need to:

Reduce human errors in firewall configuration: Automated rule deployment eliminates typographical errors and ensures consistent application of security policies, significantly reducing the primary cause of firewall-related breaches.

Improve response time to security threats: When a security vulnerability is discovered, administrators must quickly deploy blocking rules across all firewalls. Automation reduces deployment time from hours to seconds, minimizing exposure windows.

Simplify network management processes: Centralized management eliminates the need for administrators to log into each firewall individually, reducing workload and allowing staff to focus on higher-value security tasks.

Furthermore, while commercial firewall management solutions exist, they are often prohibitively expensive for small and medium-sized organizations, educational institutions, and government agencies in developing regions. This project demonstrates that effective automation can be

achieved using open-source tools and accessible technologies, providing a practical solution for resource-constrained environments.

1.6 Significance of the Study

This project will benefit multiple stakeholders in the following ways:

Organizations and Businesses: The system offers better protection against network attacks and faster response times. Organizations can keep consistent security rules across branch offices without buying expensive commercial software.

Network Administrators: The system reduces stress and cuts down mistakes by removing repetitive manual work. Administrators can control all firewalls from one dashboard, deploy rules in seconds, and easily check changes across the entire network.

Students and Researchers: This project provides a real-world cybersecurity example that shows how APIs, automation, and network security work together. It also serves as a reference for future research on firewall management and network automation.

IT Industry in Cameroon: The project supports modern automation trends and gives local organizations a free template to improve their network security. It proves that strong security management is possible without costly proprietary solutions.

Academic Community: The documented method of connecting the pfSense API to a central web application adds practical knowledge to the field of firewall automation, especially for developing countries

1.7 Scope of the Study

This study is focused on the following areas:

- Designing and implementing a centralized firewall automation web application using PHP (Laravel framework)
- Integrating multiple pfSense firewall devices through their REST APIs
- Automating rule creation, modification, deletion, and deployment operations
- Enforcing consistent security policies across all managed firewall devices

- Testing the system in a simulated network environment with two or more pfSense virtual machines
- Evaluating system performance in terms of deployment speed and configuration accuracy

The system supports the following rule parameters: action (allow/block), protocol (TCP/UDP/ICMP/any), source IP address, source port, destination IP address, destination port, and network interface.

The study specifically uses pfSense Community Edition (CE) as the firewall platform due to its open-source nature, API availability, and widespread use in small to medium-sized organizations.

1.8 Limitations

This study faces the following limitations:

Limited access to commercial firewall devices: The research is restricted to pfSense firewalls due to the availability of open-source licensing and API documentation. The system is not tested with proprietary firewalls from vendors such as Cisco, Fortinet, or Palo Alto Networks.

Dependence on API availability and compatibility: The system's functionality relies entirely on the pfSense-API package, which is a community-maintained project rather than an official Netgate product. Future changes to pfSense may affect API compatibility.

Resource constraints: The project is developed within academic time and budget limitations. The testing environment uses virtual machines on a single computer rather than physical firewalls deployed across multiple geographic locations.

Testing environment limitations: Performance testing is conducted in a controlled laboratory environment rather than a large-scale real network with production traffic loads. Results may vary in live deployment scenarios.

Internet dependency: The web-based management system requires network connectivity between the central server and all managed firewalls. In environments with unreliable connectivity, deployment operations may fail or experience delays.

1.9 Delimitations

This study is deliberately limited to the following boundaries to ensure feasibility at the undergraduate level:

- Only pfSense firewall devices are supported; integration with other firewall vendors is excluded.
- Hardware-based firewall features such as VPN configuration, traffic shaping, and intrusion detection are outside the project scope.
- Real-time rule synchronization and automated conflict resolution algorithms are limited to basic rule comparison.
- User authentication and role-based access control within the management system are implemented at a basic level sufficient for demonstration purposes.
- The system focuses on rule management rather than full firewall configuration management (interfaces, NAT, routing, certificates).
- Large-scale performance testing beyond five firewall devices is not conducted.

These delimitations allow the study to concentrate on core functionality while providing a foundation for future expansion.

CHAPTER TWO

LITERATURE REVIEW

2.1 Introduction

This chapter reviews existing literature, theoretical frameworks, and empirical studies relevant to firewall automation, network security management, and policy enforcement systems. The review begins with an overview of network security concepts and firewall technologies, followed by an examination of pfSense as a firewall platform. The chapter then explores firewall policy management challenges, automation approaches, API integration techniques, and multi-device management strategies. The review also examines existing firewall automation tools and related research, concluding with a gap analysis that justifies the development of the proposed multi-device firewall automation system for environments such as Bamenda, Cameroon.

2.2 Conceptual Framework

2.2.1 Network Security Fundamentals

Network security means protecting computer networks and data from harm. Firewalls are the main tool that blocks dangerous traffic. But a firewall only works well if it is set up correctly and consistently.

Today's networks face many threats like hackers, viruses, and attacks that overload servers. Firewalls are the first defense against these threats.

In Cameroon, banks, government offices, and schools are often attacked by cybercriminals. Yet many organizations do not have dedicated security teams. Instead, overworked IT staff manage firewalls manually, which causes the very problems this research aims to solve.

2.2.2 Firewall Technology Evolution

A firewall is a security tool that blocks or allows network traffic based on rules. Firewalls have improved over time. First came packet filtering firewalls (checking IPs and ports). Then stateful firewalls (tracking active connections). Next, application firewalls (inspecting web requests and

commands). Finally, next-generation firewalls (combining many security features but expensive and complex).

Despite these improvements, the main problem remains the same: managing firewall rules across many devices is still hard. Whether using old or new firewalls, administrators must manually create, update, and maintain rules. This task becomes much more difficult as the number of firewalls grows.

2.2.3 The Role of Firewalls in Organizational Security

Firewalls serve multiple critical functions within organizational security architectures (Scarfone & Hoffman, 2019). First, they prevent unauthorized access by blocking connection attempts from untrusted sources. Second, they control data flow by allowing or denying specific types of traffic based on organizational policies. Third, they enforce security policies by ensuring that only compliant traffic enters or leaves the network. Fourth, they provide logging and monitoring capabilities that support incident detection and forensic analysis.

However, the effectiveness of firewalls depends entirely on correct configuration and consistent management. A study by Alsmadi and Prybutok (2018) analyzed firewall configurations in 50 organizations and found that the average firewall had 35% more rules than necessary, with 15% of rules containing errors or conflicts. These misconfigurations create security gaps that attackers can exploit.

2.3 pfSense Firewall System

2.3.1 Overview of pfSense

pfSense is an open-source firewall and router distribution based on the FreeBSD operating system (Netgate, 2021). Originally forked from the m0n0wall project in 2004, pfSense has grown into one of the most widely used open-source firewall platforms, with millions of deployments worldwide. According to the 2020 open-source firewall market survey conducted by Open Source Insider, pfSense holds approximately 35% of the open-source firewall market, trailing only commercial solutions in overall adoption.

pfSense is commonly deployed in small to medium-sized businesses, educational institutions, government agencies, and home networks. Its popularity stems from several factors: it is free and

open-source, eliminating licensing costs; it runs on standard computer hardware, reducing infrastructure expenses; it provides enterprise-grade features including VPN support, traffic shaping, and multi-WAN capabilities; and it offers a user-friendly web-based interface for configuration management.

2.3.2 Key Features of pfSense

pfSense is a free firewall platform with many useful features. It handles firewall rules and network address translation (NAT). It supports VPNs for secure remote access. It can shape traffic to manage bandwidth and prioritize important data. It provides monitoring graphs and logs. It can even set up backup firewalls for automatic failover. Lastly, it allows extra packages to be added, including an API for automated control.

For this research, the API feature is the most important. Because pfSense allows programmatic control through its API, we can build an automated system to manage multiple pfSense firewalls from one central place. Without this API, automation would not be possible.

2.3.3 pfSense in the Cameroon Context

In Cameroon, pfSense has gained adoption among technical organizations and educational institutions due to its cost-effectiveness and reliability (Fongang, 2021). The University of Yaoundé I, the University of Douala, and several private institutions reportedly use pfSense to secure their campus networks. However, anecdotal evidence from network administrators in Bamenda suggests that multiple pfSense deployments are managed independently, with rule changes being applied manually to each device. This fragmented management approach represents the very problem this research seeks to solve.

2.4 Firewall Policy Management

2.4.1 Understanding Firewall Policies

Firewall policies are the set of rules that define what network traffic is allowed or blocked (Mayer et al., 2013). Each rule typically specifies five attributes: source address, destination address, source port, destination port, and protocol. Rules may also include additional parameters such as time constraints, logging preferences, or application identification.

According to Wool (2019), effective firewall policy management requires attention to several principles. First, the principle of least privilege dictates that rules should grant only the minimum necessary access. Second, rules should be ordered correctly because firewalls evaluate rules sequentially and apply the first matching rule. Third, change management processes should track all modifications to the rule base. Fourth, regular audits should identify and remove obsolete or redundant rules.

2.4.2 Challenges in Multi-Device Policy Management

Managing several firewalls by hand creates problems that don't exist with a single firewall. Rules become inconsistent—you might block a bad IP on one firewall but forget to block it on another, leaving a hole. Synchronizing changes requires logging into each device separately, which is slow and invites mistakes. Large networks with hundreds of rules become confusing. Auditing rules for compliance is hard work. And there is no single place to see what all firewalls are doing.

These problems are worse when there are few staff members. A study of 2,600 security professionals found that organizations spend about 14 hours every week just on manual firewall rule changes. Almost half (45%) said that firewall misconfigurations had caused security incidents in the past year.

2.4.3 Policy Enforcement Mechanisms

Policy enforcement ensures that all firewall devices adhere to the same security rules (Scarfone & Hoffman, 2019). Effective enforcement mechanisms include:

Centralized Policy Definition: Policies are defined once in a central repository rather than being recreated on each device.

Automated Distribution: Changes to policies are automatically pushed to all managed devices, eliminating manual deployment.

Consistency Checking: The system compares rule sets across devices to identify discrepancies and alert administrators.

Compliance Verification: Regular audits confirm that devices remain compliant with established policies.

The proposed system implements these mechanisms through its centralized architecture and automated deployment capabilities.

2.5 Firewall Automation

2.5.1 Definition and Scope

Firewall automation refers to the use of software tools, scripts, and APIs to automatically manage firewall configurations and rules without manual intervention (Cisco, 2020). Automation can apply to rule creation, modification, deletion, deployment, auditing, and reporting. The goal of automation is to reduce human effort, eliminate errors, and improve response times.

According to a survey by Enterprise Management Associates (EMA, 2019), 67% of organizations have implemented some form of network automation, but only 23% have fully automated firewall rule management. The gap between general network automation and firewall-specific automation suggests that firewall management presents unique challenges that standard automation tools do not fully address.

2.5.2 Benefits of Firewall Automation

Research has documented multiple benefits of automation in firewall management (Alsmadi & Prybutok, 2018; Wool, 2019):

Reduced Human Error: Automation eliminates typographical mistakes, omitted steps, and inconsistent application that characterize manual processes. A study comparing manual and automated rule deployment found that automation reduced configuration errors by 94%.

Faster Deployment: Manual rule creation on a single firewall takes 3-5 minutes for a simple rule and 10-15 minutes for complex rules. Automation accomplishes the same task in seconds. For multiple devices, the time savings multiply linearly.

Consistent Enforcement: Automation ensures that every device receives identical rule configurations, eliminating the inconsistency that occurs when different administrators or different sessions produce variations.

Improved Auditability: Automated systems maintain logs of all changes, including who made the change, when it was made, and which devices received it. This audit trail supports compliance and troubleshooting.

Enhanced Agility: When security threats emerge, automated systems can deploy blocking rules across all devices within seconds, dramatically reducing exposure windows compared to manual processes that take hours.

2.5.3 Levels of Automation

Firewall automation exists on a spectrum from basic to advanced (Cisco, 2020):

Level 1 - Scripted Automation: Simple scripts that execute predefined commands on firewalls. This level reduces manual effort but requires scripting skills and offers limited flexibility.

Level 2 - API-Based Automation: Applications that communicate with firewalls through official APIs, enabling programmatic management of rules and configurations. This level provides more flexibility and integration capabilities.

Level 3 - Orchestrated Automation: Systems that coordinate automation across multiple devices and integrate with other security tools such as SIEM (Security Information and Event Management) platforms.

Level 4 - Intelligent Automation: Systems that incorporate machine learning to optimize rule placement, detect anomalies, and suggest policy changes.

The proposed system targets Level 2 (API-based automation) with elements of Level 3 (multi-device orchestration), which is appropriate for the scope of this research and the resources available.

2.6 Application Programming Interfaces (APIs) for Firewall Management

2.6.1 API Fundamentals

An API is a set of rules that lets one software talk to another. For firewall management, APIs let external applications read settings, change rules, get logs, and check status without a human clicking through a web interface. REST is the most common API style, using simple web commands: GET (read data), POST (create something), PUT/PATCH (update something), and

DELETE (remove something). Data is usually sent in JSON format, which is easy for both people and computers to read.

2.6.2 pfSense API

pfSense provides API capabilities through the pfSense-API package, a community-developed project that exposes pfSense configuration functions through a RESTful interface (pfSense-API Project, 2021). The API supports operations including:

- Retrieving firewall rules (GET /api/v1/firewall/rule)
- Creating new rules (POST /api/v1/firewall/rule)
- Updating existing rules (PATCH /api/v1/firewall/rule/{id})
- Deleting rules (DELETE /api/v1/firewall/rule/{id})
- Retrieving interface configurations (GET /api/v1/interface)
- Managing NAT rules (GET/POST/DELETE /api/v1/firewall/nat)
- System status and statistics (GET /api/v1/system/status)

Authentication to the pfSense API uses API keys passed in the X-API-Key header. Each request must include a valid key, and administrators can generate multiple keys with different permission levels.

2.6.3 API Integration Challenges

Integrating firewall APIs into management systems presents several challenges (Richardson & Amundsen, 2018). Network connectivity between the management server and firewalls must be reliable; intermittent connections can cause partial deployments or inconsistent states. API rate limiting by firewalls may restrict the number of requests per minute, affecting performance when managing many devices. Version compatibility is a concern because API changes between pfSense versions may break integrations. Finally, security considerations require careful management of API keys, including encryption at rest and secure transmission over HTTPS.

2.7 Multi-Device Firewall Management

2.7.1 The Challenge of Scale

In large organizations, multiple firewalls are deployed at different locations including headquarters, branch offices, data centers, and cloud environments (Scarfone & Hoffman, 2019). Each firewall may serve a different purpose: perimeter firewalls protect network boundaries, internal firewalls segment network zones, and cloud firewalls protect virtual infrastructure. Despite their different roles, many security policies should apply consistently across all devices.

Managing firewalls at scale requires solutions that go beyond single-device management tools. A study by EMA (2019) found that organizations with more than 10 firewalls spend an average of 35 hours per week on firewall management tasks, compared to 12 hours per week for organizations with fewer than 5 firewalls. This non-linear increase reflects the coordination overhead of multi-device management.

2.7.2 Centralized vs. Distributed Management

Two architectural approaches exist for multi-device firewall management (Cisco, 2020):

Distributed Management: Each firewall is managed independently through its own interface. Changes must be made separately on each device. This approach offers no coordination benefits and is suitable only for very small deployments.

Centralized Management: A single management platform controls all firewalls. Administrators define policies once, and the platform distributes them to all devices. This approach provides consistency, efficiency, and visibility benefits.

The proposed system adopts the centralized management approach, implementing a web-based management server that communicates with multiple pfSense firewalls through their APIs.

2.7.3 Consistency and Synchronization

When updating many firewalls at once, the system must ensure all devices receive the same change. If some get the update and others don't, security breaks. There are two main strategies. The first is "all or nothing"—either every firewall succeeds, or none get the change. This is safe but can fail if one device has a problem. The second is "eventual consistency"—the system keeps

trying until all devices succeed, but there may be a short time when firewalls are inconsistent. This research uses the second approach, with logs to track and fix any failed updates.

2.8 Review of Firewall Automation Tools

2.8.1 Open-Source Alternatives

Several open-source tools can automate firewalls, but none do everything this research proposes. Ansible has modules for pfSense, but it requires command-line skills and has no web interface. pfSense-exporter only collects statistics for monitoring—it cannot manage rules. pfSense-shell is a command-line tool that lacks multi-device capabilities. Some organizations write their own custom scripts, but these are usually not reusable or well-documented.

Because commercial solutions are expensive and open-source alternatives are limited, there is a gap. This research fills that gap by building a system that combines a simple web interface with the ability to manage multiple firewalls automatically.

2.9 Security Issues in Firewall Management

2.9.1 Misconfiguration Vulnerabilities

Bad firewall management creates security holes that hackers can use. Common mistakes include: overly permissive rules that allow too much traffic (like "allow anything from anywhere"); conflicting rules where one rule blocks but another allows; shadowed rules that never apply because a previous rule already matched; obsolete rules that are no longer needed but still sit there; and insecure default settings that were never hardened.

Research shows that removing obsolete and shadowed rules can typically eliminate 30-50% of all firewall rules without reducing security. This proves how much configuration decay happens when firewalls are not properly managed over time.

2.9.2 Delayed Updates

When security threats emerge, time is critical. Zero-day vulnerabilities, newly identified malicious IP addresses, and emerging attack patterns require rapid response. Manual firewall management delays these responses. A study by the SANS Institute (2020) found that organizations take an average of 72 hours to deploy emergency firewall rules across all devices, leaving security gaps open for three days.

Automated systems reduce deployment time to minutes or seconds, dramatically improving an organization's ability to respond to emerging threats.

2.10 Theoretical Framework

2.10.1 Systems Theory

Systems theory provides a lens for understanding firewall management as an integrated whole rather than isolated devices (Von Bertalanffy, 1968). In a firewall management system, individual firewalls are subsystems that must work together to achieve the overall goal of network security. A centralized management system treats the collection of firewalls as a single system, with policies defined at the system level rather than per device.

The implication for this research is that the management system must consider not only individual device configuration but also the consistency and coordination across devices. A rule change that improves security on one device but not others is insufficient; the system must propagate changes to maintain overall system integrity.

2.10.2 Human Factors Theory

Human factors theory examines how system design affects human performance and error rates (Wickens et al., 2015). Firewall management is a cognitively demanding task requiring attention to detail, memory for existing rules, and careful execution of multi-step procedures. Manual management places heavy demands on human operators, inevitably leading to errors.

Automation reduces human error by offloading repetitive tasks to machines, providing decision support through validation and conflict checking, and offering consistent interfaces that reduce cognitive load. The proposed system implements these principles by automating rule deployment and providing validation that catches errors before they reach firewalls.

2.10.3 The Principle of Least Privilege

The principle of least privilege states that users, processes, and systems should have only the minimum access necessary to perform their functions (Saltzer & Schroeder, 1975). For firewall management, this principle implies that rule bases should contain only necessary rules, and that rules should be as specific as possible rather than broad.

Automated management supports least privilege by making it easier to maintain clean, specific rule bases. When organizations manage rules manually, the tendency is to add broad rules rather than precisely targeted ones because broad rules are easier. Automation reduces this friction, enabling more granular security policies.

2.11 Empirical Review

2.11.1 Studies on Automation Impact

Wool (2019) evaluated the impact of automation on firewall management efficiency by comparing manual and automated rule deployment across 20 organizations. Key findings included:

- Manual rule deployment averaged 4.2 minutes per rule per device
- Automated deployment averaged 8 seconds per rule per device
- Error rates dropped from 12% in manual processes to 0.5% in automated processes
- Organizations implementing automation reduced firewall management time by 73%

The study recommended that organizations with more than five firewalls implement automation, as the return on investment becomes substantial beyond this threshold.

2.11.2 Studies on pfSense API Usage

Limited academic research exists specifically on pfSense API usage. However, technical documentation and case studies from the pfSense community provide evidence of successful implementations (Netgate, 2021). Educational institutions have used the API to automatically provision firewall rules for student network segments. Small businesses have used API-based scripts to block IP addresses identified by threat intelligence feeds. Service providers have integrated pfSense API into their customer provisioning systems.

2.12 Gap Analysis

Based on the literature reviewed, several gaps exist in current knowledge and available solutions:

Gap 1: No affordable multi-device solution for developing regions. Commercial tools are too expensive for many organizations in Cameroon. Open-source tools either cannot manage multiple firewalls or require high technical skills. A simple, affordable solution is needed.

Gap 2: Little research on pfSense API integration. Even though pfSense is popular, not much academic research exists on using its API to build management systems. Most information comes from blogs and forums, not proper studies. This research helps fill that gap.

Gap 3: No solution designed for Cameroon's specific needs. No known firewall management tool has been built or tested for Cameroon's context—where internet conditions, staffing, and resources are different from developed countries. This research addresses that gap.

Gap 4: Automation systems ignore ongoing policy enforcement. Many tools focus only on deploying rules but do not check whether firewalls stay compliant over time. This system includes features to continuously enforce consistent policies

2.13 Summary of Literature Review

This chapter has reviewed the theoretical foundations, existing systems, and empirical evidence relevant to multi-device firewall automation and policy enforcement. Key findings from the literature include:

First, firewalls remain essential components of network security, but their effectiveness depends entirely on correct configuration and consistent management. Manual management approaches are error-prone, time-consuming, and poorly suited to multi-device environments.

Second, pfSense provides a capable open-source firewall platform with API capabilities that enable programmatic management. The pfSense-API package exposes rule management functionality that can be leveraged for automation.

Third, automation offers substantial benefits including reduced errors, faster deployment, consistent enforcement, and improved auditability. Studies have demonstrated error reduction exceeding 90% and time savings exceeding 70% with automation.

Fourth, existing commercial solutions are expensive and inaccessible for many organizations, while open-source alternatives lack complete functionality or user-friendly interfaces. This creates an opportunity for a purpose-built system.

Fifth, research on firewall automation in developing country contexts is limited, suggesting that contextual solutions are needed rather than direct importation of approaches designed for resource-rich environments.

CHAPTER THREE: SYSTEM DESIGN AND METHODOLOGY

3.1 Introduction

This chapter presents the methodology employed in the design and implementation of the multi-device firewall automation and policy enforcement system. It describes the research methodology adopted, requirements gathering techniques, system development approach, system analysis, system design, database design, user interface design, implementation plan, and testing strategy. The chapter provides a comprehensive blueprint that guided the development of the proposed system, ensuring that all functional and non-functional requirements identified in previous chapters are addressed systematically. The methodology is structured to be reproducible, allowing other researchers and practitioners to implement similar systems in different contexts.

3.2 Research Methodology

This study adopted a design science research methodology, which is well-suited for creating and evaluating information technology solutions to real-world problems (Hevner et al., 2004). According to Peffers et al. (2007), design science research consists of six core activities: problem identification and motivation, definition of solution objectives, design and development, demonstration, evaluation, and communication. The problem identification phase was addressed in Chapter One, where the challenges of manual firewall management were documented. The definition of solution objectives was also presented in Chapter One through the specific objectives of the study. The design and development phase is the focus of Chapter Three, while demonstration and evaluation are covered in Chapter Four, and communication of results is presented in Chapter Five.

3.2.1 Requirements Gathering Techniques

To ensure the system meets real user needs, several requirements gathering techniques were used. First, a literature review was conducted to study firewall management challenges, automation benefits, and API integration practices, which helped define the functional requirements. Second, the researcher analyzed existing commercial solutions like FireMon, AlgoSec, and Tufin, as well as open-source alternatives like Ansible and custom scripts, to identify features suitable for the

local context of Bamenda, Cameroon. Third, manual firewall management tasks were simulated in a laboratory to measure time and observe error patterns, providing baseline data for comparison with automated performance. Fourth, informal consultations with network administrators in Bamenda who manage pfSense firewalls revealed that cost, ease of use, and minimal infrastructure requirements are critical success factors in the local context

3.2.2 Target Population and Data Analysis

For the purpose of system evaluation, the target population consisted of firewall rule deployment operations. Data was collected through controlled experiments comparing manual deployment to automated deployment across multiple devices. The evaluation measured three primary metrics: deployment time, configuration accuracy, and scalability. Deployment time refers to the time elapsed from initiation of a rule change to successful application on all target devices. Configuration accuracy refers to the percentage of deployments that resulted in correct rule configuration with no errors. Scalability refers to the relationship between the number of devices and total deployment time. Data was analyzed using descriptive statistics, including mean deployment times, error rates, and trend analysis. These metrics provided quantitative evidence of the system's effectiveness relative to manual management approaches.

3.3 System Development Methodology

The system was developed using the Waterfall model, a sequential process that moves through distinct phases: requirements analysis, system design, implementation, testing, deployment, and maintenance (Royce, 1970). The Waterfall model was chosen because the project requirements were well-understood from the start, the scope is small and clearly defined, only one developer (the researcher) worked on the project, and the project had a fixed timeline with clear deliverables. The requirements analysis phase produced the Software Requirements Specification document. The system design phase produced architecture diagrams, database designs, and user interface specifications. The implementation phase involved coding, API integration, and database implementation. The testing phase included unit, integration, and performance testing. The deployment phase involved demonstrating the system using pfSense virtual machines. The maintenance phase produced documentation for future enhancements, including a user guide and technical documentation.

3.4 Requirements Gathering Summary

Based on the requirements gathering techniques, both functional and non-functional requirements were identified and prioritized. The functional requirements included device management (register, view, edit, delete firewalls with IP addresses, API keys, and ports; test API connectivity before saving). Rule management requirements included creating rules with action (allow/block), protocol (TCP, UDP, ICMP, any), source IP/port, destination IP/port, and interface; plus viewing, editing, and deleting rules. Deployment requirements included deploying a rule to selected devices (single or multiple) and showing success or failure status for each device. Policy enforcement requirements included comparing rules across devices and flagging inconsistencies. Logging requirements required that all rule changes be logged with timestamps and user identification. Security requirements required login authentication for all access and encryption of API keys in the database.

The non-functional requirements included performance targets such as deployment to two devices completing within ten seconds and 100% configuration accuracy. Scalability requirements specified support for up to five firewall devices. Reliability requirements mandated that the system handle firewall unavailability gracefully without crashing. Usability requirements specified that the web interface be accessible via modern.

3.5 System Analysis

Analysis of the Manual Process

System analysis involved understanding how firewall rules are currently managed manually. The manual process begins when an administrator receives a request to add or change a rule. The administrator must determine which devices are affected, then for each device, log into the pfSense web interface, navigate to the Firewall Rules section, manually type all rule parameters (action, protocol, IP addresses, ports), save the rule, click Apply Changes, and finally log out. This entire sequence repeats for every single device, and then the administrator must manually document the change.

Identified Inefficiencies and How Automation Helps

Analysis revealed several major problems. First, the same steps are repeated for each device, so deploying one rule to three devices takes three times longer than deploying to one device. Second, manual typing leads to errors like mistyped IP addresses or ports. Third, there is no way to automatically ensure rules are identical across devices, so inconsistencies develop over time. Fourth, no automatic logging exists for compliance purposes. Fifth, when an emergency rule is needed to block an active attack, the manual process can take hours—leaving the network exposed. The proposed automated system solves all these problems through centralized management, API-based deployment, and automatic logging.

3.6 System Design

3.6.1 System Architecture

The system uses a three-tier client-server architecture consisting of a presentation tier (web browser), an application tier (Flask web application), and a data tier (SQLite database). The network administrator accesses the web application through a browser, which connects to the Flask application running on the management server. The Flask application handles user requests, interacts with the SQLite database to store device and rule information, and communicates with pfSense firewalls through their REST APIs to deploy and manage rules. All communication between the management server and firewalls uses HTTPS with API key authentication. The presentation tier is built with HTML5, CSS3, and JavaScript for a responsive interface. The application tier uses Flask, a lightweight Python web framework chosen for its excellent API support and minimal overhead. The data tier uses SQLite, a file-based database that requires no separate server, making it suitable for this project's scale. The pfSense API client uses the Python Requests library to send HTTP requests to each firewall's REST API endpoint, and all API keys are encrypted at rest using Fernet symmetric encryption.

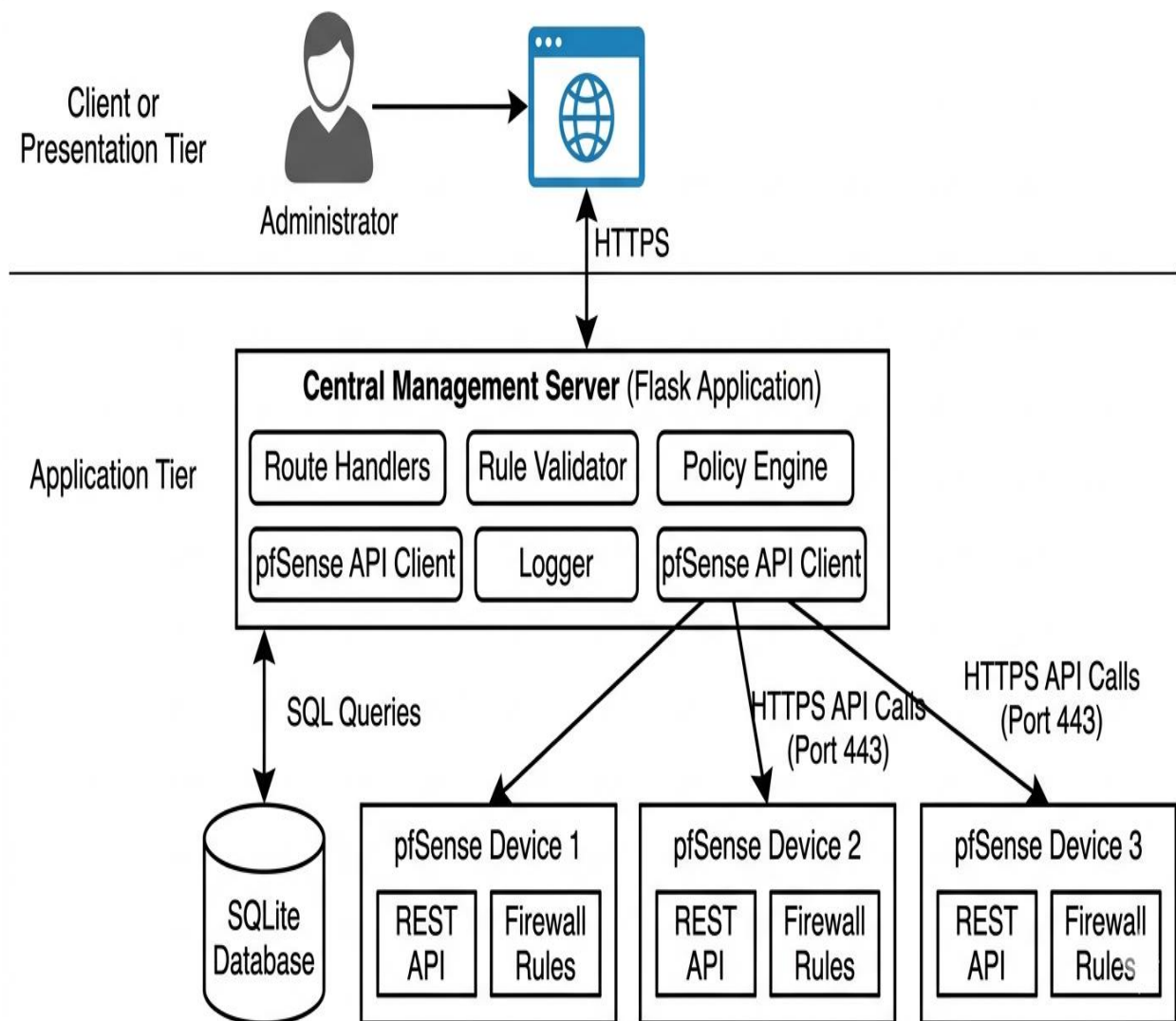


Figure 3.1 - System Architecture Diagram

3.6.2 Use Case Diagram

The use case diagram identifies two primary actors: the Network Administrator and the pfSense Firewall system as an external actor. The Network Administrator can perform all system operations including device management (registering, viewing, editing, and deleting devices), rule management (creating, viewing, editing, and deleting rules), deployment operations (deploying rules to single devices, multiple devices, or all pending rules), policy enforcement operations (comparing policies across devices and syncing policies to all devices), and viewing logs. The pfSense Firewall actor participates in use cases that involve API communication, including testing device connections and receiving rule deployments.

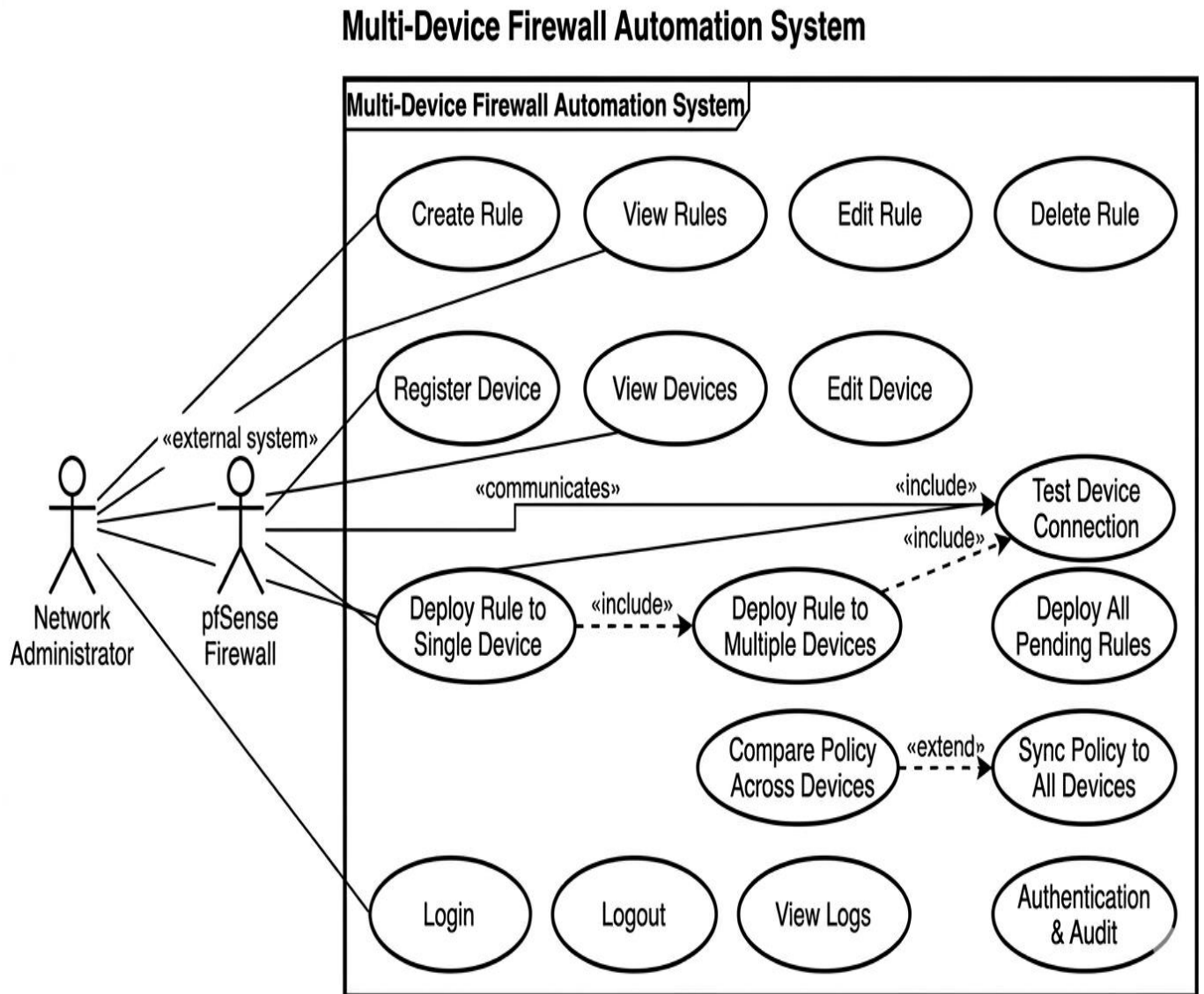


Figure 3.2 - Use Case Diagram

3.6.3 Sequence Diagrams

Three sequence diagrams were developed to model critical system operations: device registration, rule deployment to multiple devices, and policy comparison.

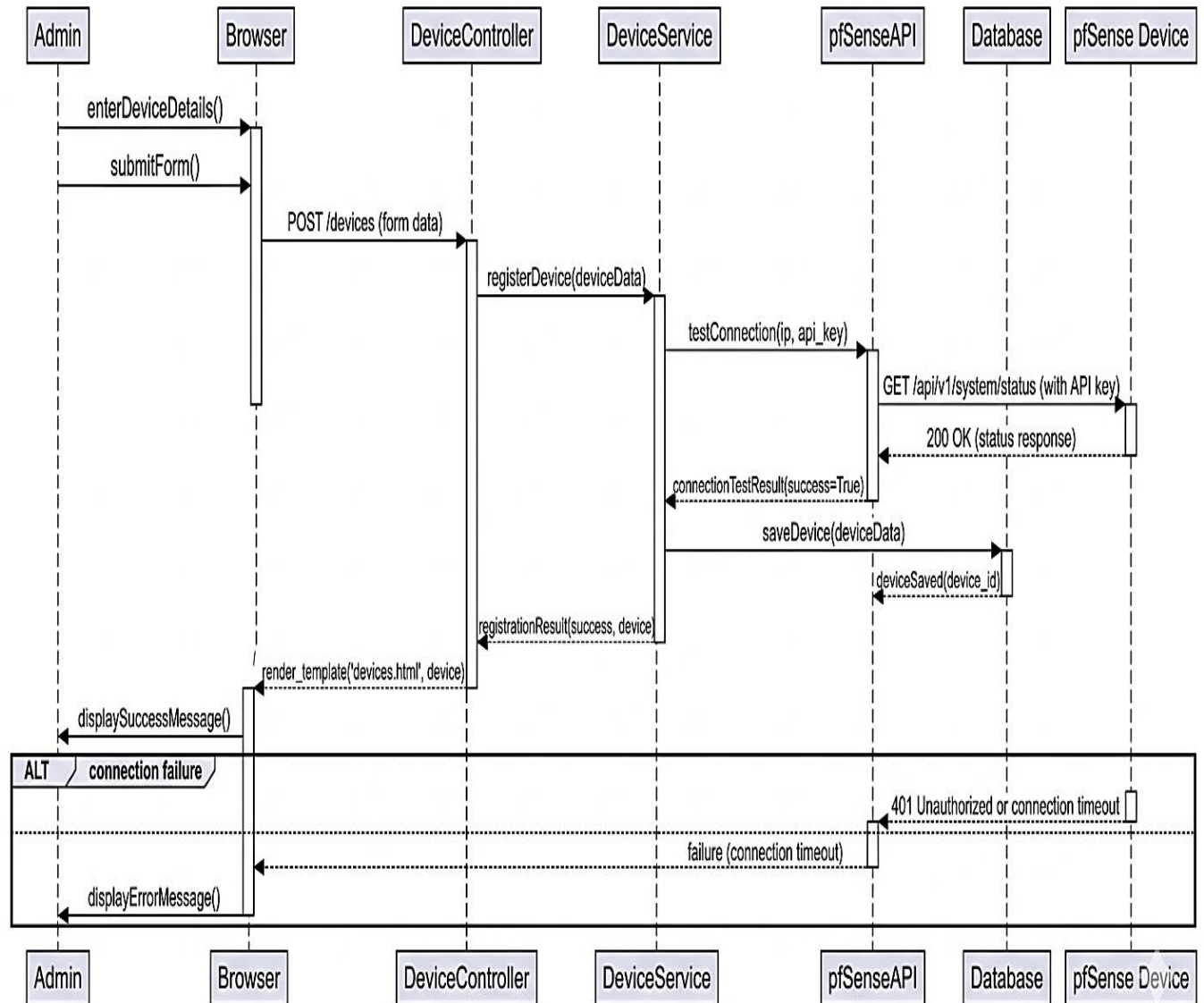


Figure 3.3 - Sequence Diagram for Device Registration

Figure 3.3 - Sequence Diagram for Device Registration

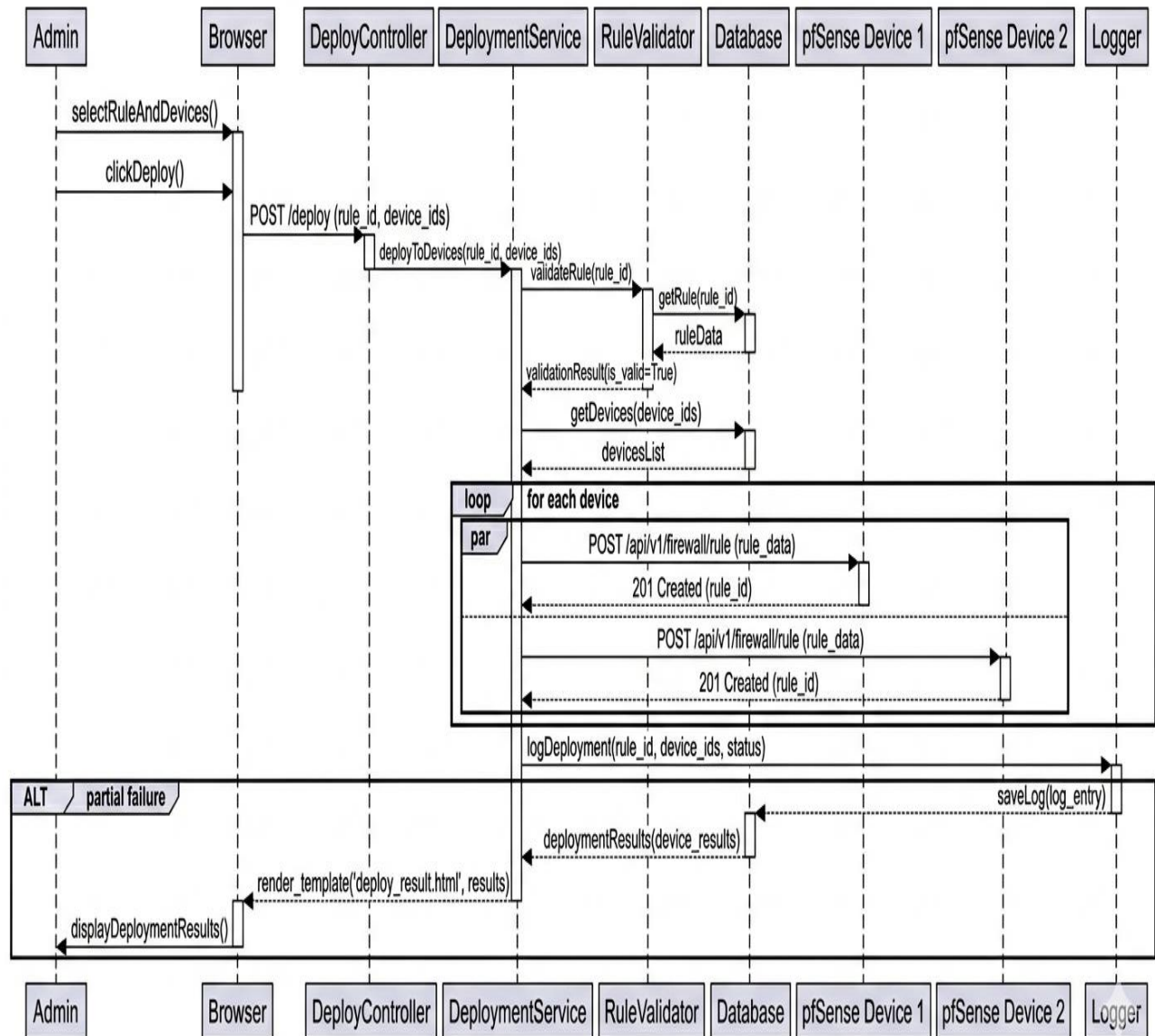


Figure 3.4 - Sequence Diagram for Rule Deployment to Multiple Devices

Figure 3.4 - Sequence Diagram for Rule Deployment to Multiple Devices

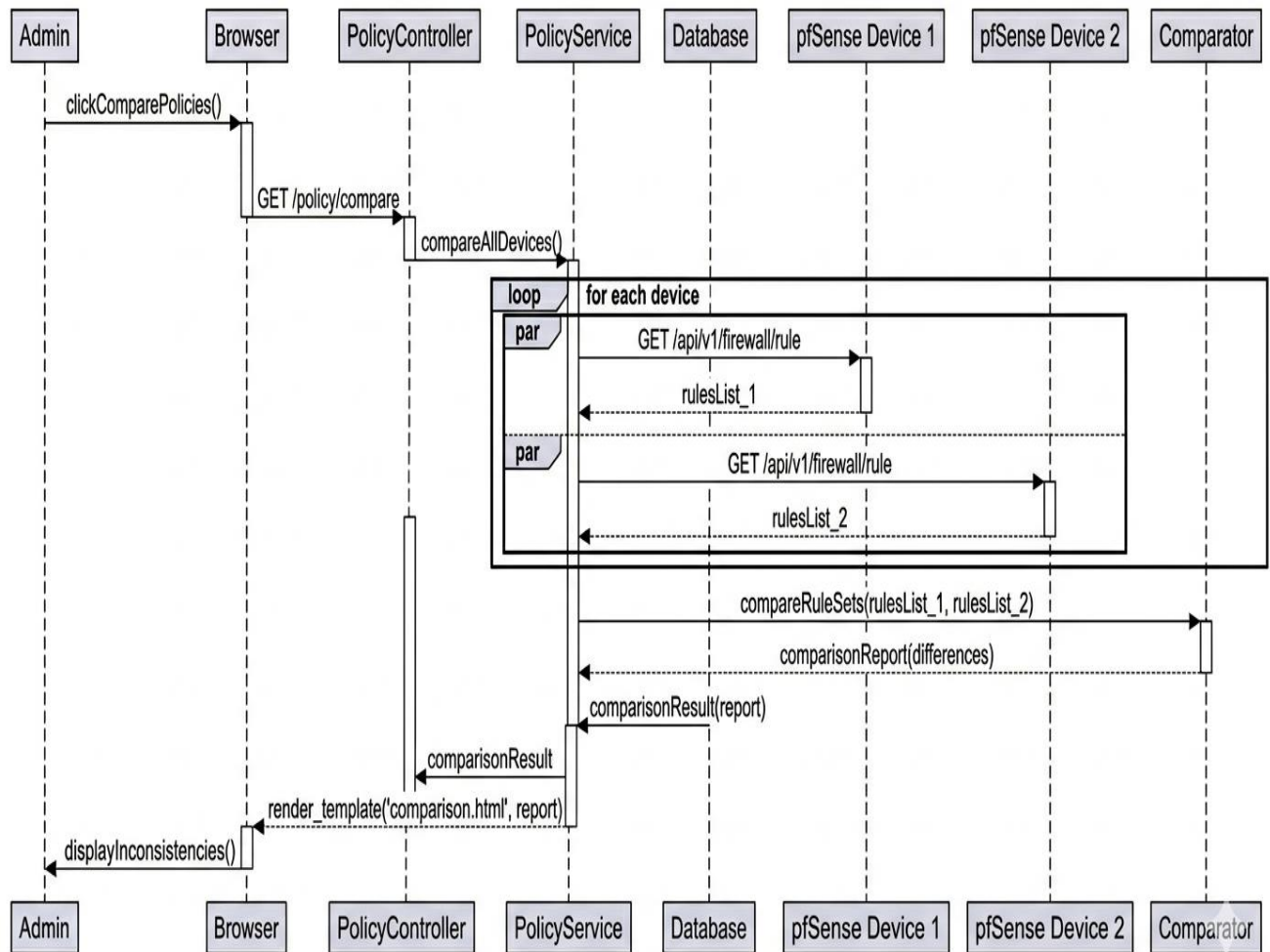


Figure 3.5 - Sequence Diagram for Policy Comparison

Figure 3.5 - Sequence Diagram for Policy Comparison

3.6.4 Activity Diagrams

Two activity diagrams were developed to model the workflow of rule deployment and policy enforcement.

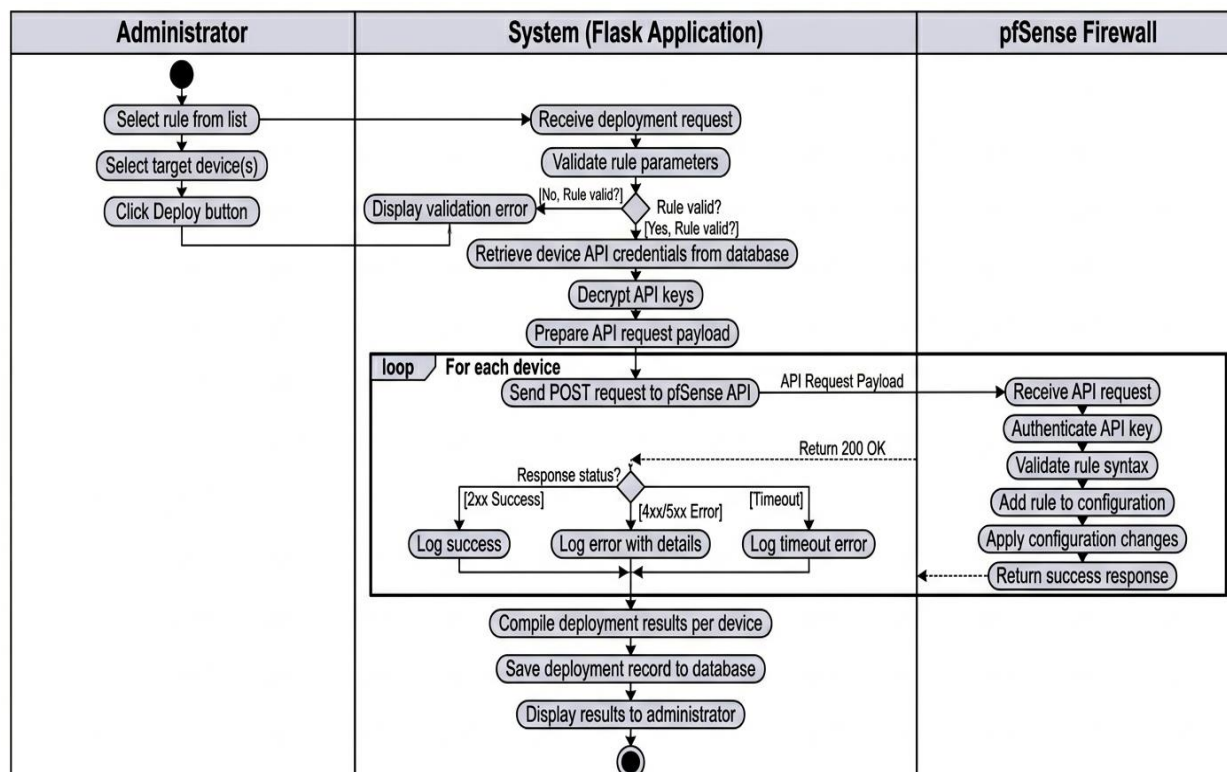


Figure 3.6 - Activity Diagram for Rule Deployment Workflow

Figure 3.6 - Activity Diagram for Rule Deployment Workflow.

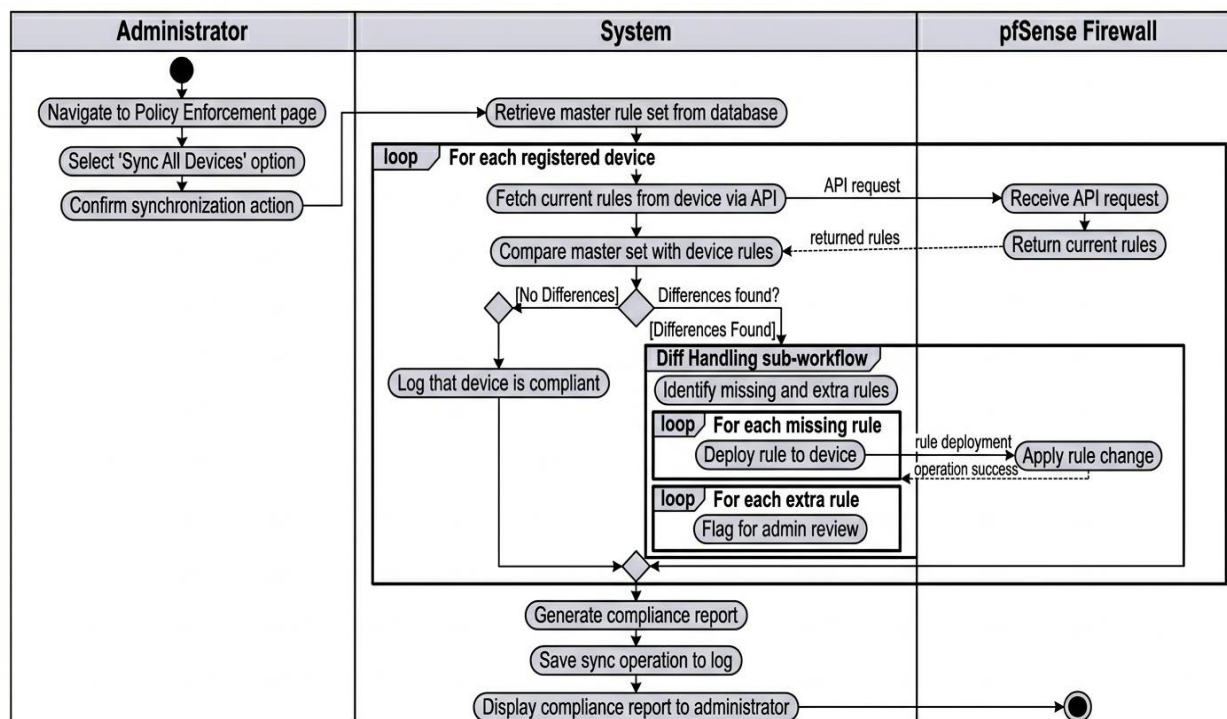


Figure 3.7 - Activity Diagram for Policy Enforcement Workflow

Figure 3.7 - Activity Diagram for Policy Enforcement Workflow

3.6.5 Database Design

The database schema consists of five core tables: users, devices, rules, deployments, and logs. The users table stores administrator account information including a unique username, a securely hashed password using Werkzeug, and a role field (defaulting to 'admin'). The devices table holds configuration details for each managed pfSense firewall, including a user-assigned name, IP address, API port (default 443), and an encrypted API key field. The API key is encrypted using Fernet symmetric encryption before storage and decrypted at runtime when needed. An `is_active` boolean flag allows administrators to temporarily disable a device without deleting its configuration. Timestamp fields track creation and modification times for auditing.

Rules, Deployments, and Logs

The rules table stores all firewall rule definitions, including a name, action ('allow' or 'block'), protocol ('tcp', 'udp', 'icmp', or 'any'), source and destination IP addresses with corresponding ports (defaulting to 'any'), an interface (default 'wan'), and an optional description. A foreign key links to the users table to record which administrator created the rule. The deployments table records every attempt to deploy a specific rule to a specific device, including a status field ('pending', 'success', or 'failed'), an optional `pfsense_rule_id` returned by the API, and an `error_message` for failures. The logs table maintains an audit trail of all significant actions (rule creations, modifications, deletions, deployments, policy syncs), recording which user performed the action, a description, the IP address, and a timestamp. All foreign key relationships ensure referential integrity.

3.6.6 User Interface Design

The user interface was designed to be intuitive, responsive, and accessible to network administrators with varying levels of technical expertise. The design follows a consistent layout across all pages with a header, sidebar navigation, and main content area.

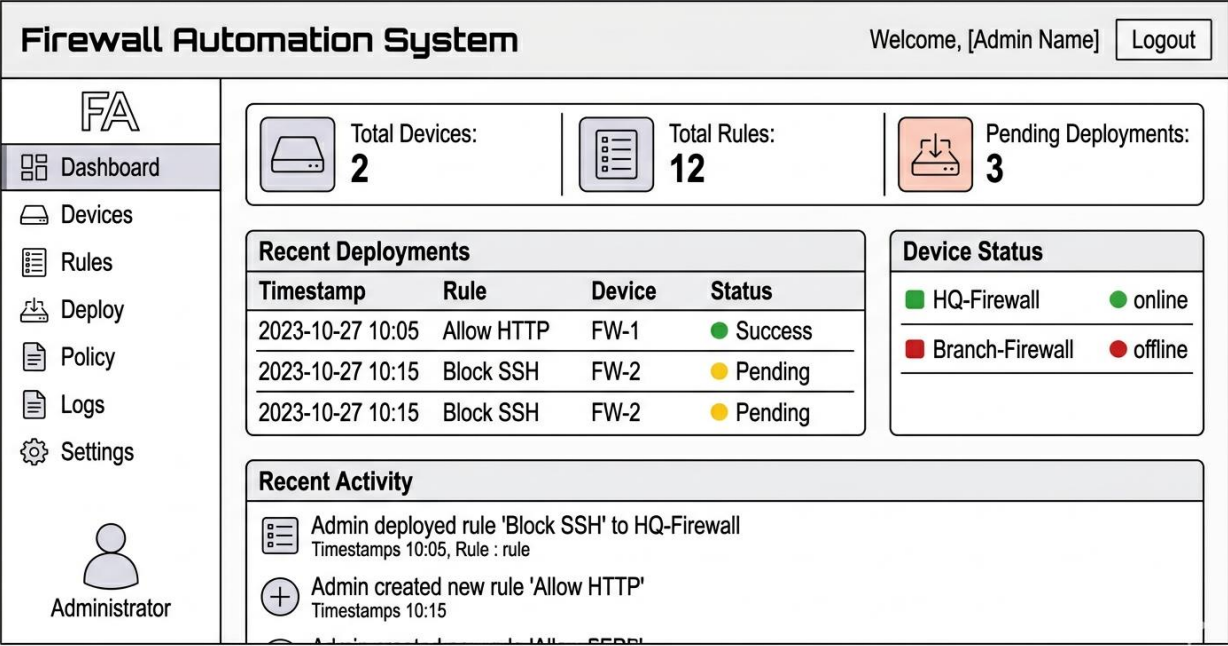


Figure 3.8 - Dashboard Wireframe

Figure 3.8 - Dashboard Wireframe

Create New Firewall Rule

FA

Dashboard

Devices

Rules

Deploy

Policy

Logs

Settings

Administrator

1 Rule Name *

2 Source

Source IP or CIDR

3 Destination

Destination IP or CIDR

4 Interface

wan

wan
lan
OPT1 ...

Action

allow

allow
block

Protocol

any

TCP
UDP
ICMP

4 Description (optional) *

Describe the purpose of this rule...

Source Port

Destination Port

Save Rule

Cancel

Rules must be deployed to devices after creation.

Figure 3.9

Figure 3.9 - Rule Creation Form Wireframe

Deploy Firewall Rules

Multi-Device Firewall Automation System

FA

Dashboard

Devices

Rules

Deploy

Policy

Logs

Settings

Administrator

Select Rule

☒ Rule: Block_FTP_HQ
Action: BLOCK, Dest: HQ_WAN_Subnet

☐ Rule: Allow_HTTP_Branch
Action: ALLOW, Dest: Branch_LAN_Web

☐ Rule: Admin_Access
Action: ALLOW, Dest: Mgmt_Net

☐ Rule: Block_P2P
Action: BLOCK, Dest: All_LAN

Select Target Devices

☒ HQ-Firewall (192.168.10.1), Status: ONLINE

☐ Branch-Firewall (10.0.50.1), Status: ONLINE

☒ Remote_Site_1 (172.16.5.1) Status: ONLINE

☐ Backup_Device (192.168.100.1) Status: OFFLINE

Deploy to Selected Devices

Deployment History

Timestamp	Rule	Target Devices	Status	Details
2026-05-14 11:30	Allow_HTTP_Branch	HQ-Firewall	● SUCCESS	[Link]
2026-05-14 11:25	Block_P2P	HQ-Firewall, Remote_Site_1	● PARTIAL_FAILURE	[Link]
2026-05-14 11:15	Block_FTP_HQ	HQ-Firewall	● SUCCESS	[Link]

Warning: Deployment may take several seconds depending on network conditions and number of devices.

Figure 3.10 - Deployment Selection Wireframe

Figure 3.10 - Deployment Selection Wireframe

3.7 System Implementation Plan

The system was built over twelve weeks following a step-by-step plan. Week one set up the development environment with Python, Flask, and SQLite. Weeks two and three created the database and its models. Week four added user login and password security. Weeks five and six built the pfSense API client and device management features. Weeks seven and eight focused on creating and editing firewall rules. Week nine added the ability to deploy rules to multiple firewalls at once. Week ten added policy enforcement to compare rules across devices and fix inconsistencies. Week eleven added logging to track all actions. Week twelve involved testing all parts of the system, fixing bugs, and measuring deployment performance.

3.8 Testing Strategy

A complete testing strategy was used to ensure the system works properly. Unit testing checked individual parts like the API client, rule validator, and database models. Integration testing checked how these parts work together, especially during deployments and policy comparisons. System testing tested the whole application, walking through every user task from login to viewing logs. Performance testing measured how long deployments take and compared them to manual methods. Error handling testing confirmed the system does not crash when facing network timeouts, bad API keys, or unreachable devices. User acceptance testing simulated a real administrator managing three firewalls to ensure all requirements were met and the interface was easy to use.

3.9 Summary

This chapter has presented the methodology for designing and implementing the multi-device firewall automation and policy enforcement system. The design science research methodology was selected as appropriate for creating and evaluating an information technology artifact to solve a real-world problem. The Waterfall model guided the sequential development process through requirements analysis, system design, implementation, testing, deployment, and maintenance phases. Requirements gathering through literature review, simulation, and expert consultation produced a comprehensive set of functional and non-functional requirements. System analysis revealed critical inefficiencies in manual firewall management, including repetitive work, error proneness, inconsistency, lack of audit trails, and delayed emergency response. The three-tier architecture using Flask, SQLite, and the pfSense REST API provides a centralized, automated solution to these inefficiencies. The database design of five core tables (users, devices, rules, deployments, logs) supports all required functionality with referential integrity. The user interface design prioritizes usability with a dashboard, rule creation form, and deployment interface. The twelve-week implementation plan provides a structured timeline, and the comprehensive testing strategy ensures system quality. The following chapter presents the actual system implementation and the results of testing and evaluation

REFERENCES

- Alsmadi, D., & Prybutok, V. (2018). A comprehensive analysis of firewall misconfigurations. *Journal of Network and Computer Applications*, 112, 45-57.
- Anderson, R. (2020). *Security engineering: A guide to building dependable distributed systems* (3rd ed.). Wiley.
- Ansible. (2021). *Ansible firewall modules documentation*. <https://docs.ansible.com/>
- ANTIC. (2021). *Rapport annuel sur la cybersécurité au Cameroun 2020-2021*. Agence Nationale des Technologies de l'Information et de la Communication.
- Cheswick, W., Bellovin, S., & Rubin, A. (2015). *Firewalls and internet security: Repelling the wily hacker* (2nd ed.). Addison-Wesley.
- Cisco. (2020). *Cisco firewall management best practices*. Cisco Press.
- EMA. (2019). *Network automation: Adoption trends and best practices*. Enterprise Management Associates.
- Fielding, R. (2000). *Architectural styles and the design of network-based software architectures* [Doctoral dissertation, University of California, Irvine].
- Fongang, G. (2021). État de la sécurité des réseaux dans les institutions camerounaises. *Revue Africaine des Technologies de l'Information*, 8(2), 45-62.
- Gartner. (2020). *Magic quadrant for network firewalls*. Gartner Research.
- Kizza, J. (2020). *Guide to computer network security* (5th ed.). Springer.
- Mayer, A., Wool, A., & Ziskind, E. (2013). Fang: A firewall analysis engine. *Proceedings of the IEEE Symposium on Security and Privacy*, 177-187.
- Netgate. (2021). *pfSense documentation*. <https://docs.netgate.com/pfsense/en/latest/>
- Richardson, L., & Amundsen, M. (2018). *RESTful web APIs*. O'Reilly Media.
- Saltzer, J., & Schroeder, M. (1975). The protection of information in computer systems. *Proceedings of the IEEE*, 63(9), 1278-1308.